

NYCU-EE IC LAB – Autumn 2025

Lab02

Design: Sudoku (SUDOKU)

Data Preparation

1. Extract test data from TA's directory:
`% tar xvf ~iclabTA01/Lab02_2025A.tar`
2. The extracted LAB directory contains:
 - a. 00_TESTBED
 - b. 01_RTL
 - c. 02_SYN
 - d. 03_GATE
 - e. 09_SUBMIT

Design Description

Sudoku is a logic-based, combinatorial number-placement puzzle. It is played on a 9x9 grid, divided into nine 3x3 boxes. The puzzle starts with some cells pre-filled with numbers, and the player must fill in the rest with digits from 1 to 9 so that each row, column, and box contain all nine digits exactly once. Using logical deduction ensures no number is repeated in any row, column, or box.

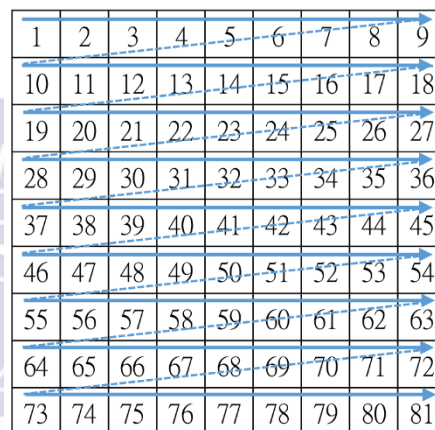
5	3			7				
6			1	9	5			
	9	8					6	
8				6				3
4			8		3			1
7				2				6
	6					2	8	
			4	1	9			5
				8			7	9

5	3	4	6	7	8	9	1	2
6	7	2	1	9	5	3	4	8
1	9	8	3	4	2	5	6	7
8	5	9	7	6	1	4	2	3
4	2	6	8	5	3	7	9	1
7	1	3	9	2	4	8	5	6
9	6	1	5	3	7	2	8	4
2	8	7	4	1	9	6	3	5
3	4	5	2	8	6	1	7	9

Figure 1. A typical Sudoku puzzle and the solution to it.

In this lab, you'll be designing a solver for Sudoku. This solver will receive a 9x9 Sudoku puzzle and transmit the solution from it.

Your Sudoku solver receives two input signals, “**in_valid**” and “**in**”, for 81 consecutive cycles. When “**in_valid**” is high, it indicates the “**in**” signal is valid. The signal of “**in**” provides the input digits for a 9x9 Sudoku puzzle in raster scan order shown in Fig. 2. Digits from 1 to 9 represent pre-filled cell (called “givens”), while a digit of 0 denotes a blank cell; the number of blank cells is random.



1	2	3	4	5	6	7	8	9
10	11	12	13	14	15	16	17	18
19	20	21	22	23	24	25	26	27
28	29	30	31	32	33	34	35	36
37	38	39	40	41	42	43	44	45
46	47	48	49	50	51	52	53	54
55	56	57	58	59	60	61	62	63
64	65	66	67	68	69	70	71	72
73	74	75	76	77	78	79	80	81

Figure 2. The schematic of raster scan order; numbers represent the order.

Fortunately, solution to the Sudoku puzzle is unique, so you don't need to consider the case of several solutions. Your Sudoku solver outputs two signals, “**out_valid**” and “**out**”, for 81 consecutive cycles. When “**out_valid**” is high, it indicates the “**out**” signal is valid. The signal of “**out**” is the solution to the Sudoku puzzle with digits from 1 to 9 in raster scan order.

The following is the example of Sudoku solver to solve the solution to the Sudoku puzzle. The cell with 0 represents the blank. The Sudoku solver you design select a number from 1 to 9 to fill in a cell marked with 0. The chosen number must satisfy the condition that the row, column, and box it belongs to all contain the number 1 to 9 without omission or repetition, making it a valid number. By sequentially filling all cell marked with 0 with the correct number, the Sudoku puzzle is solved.

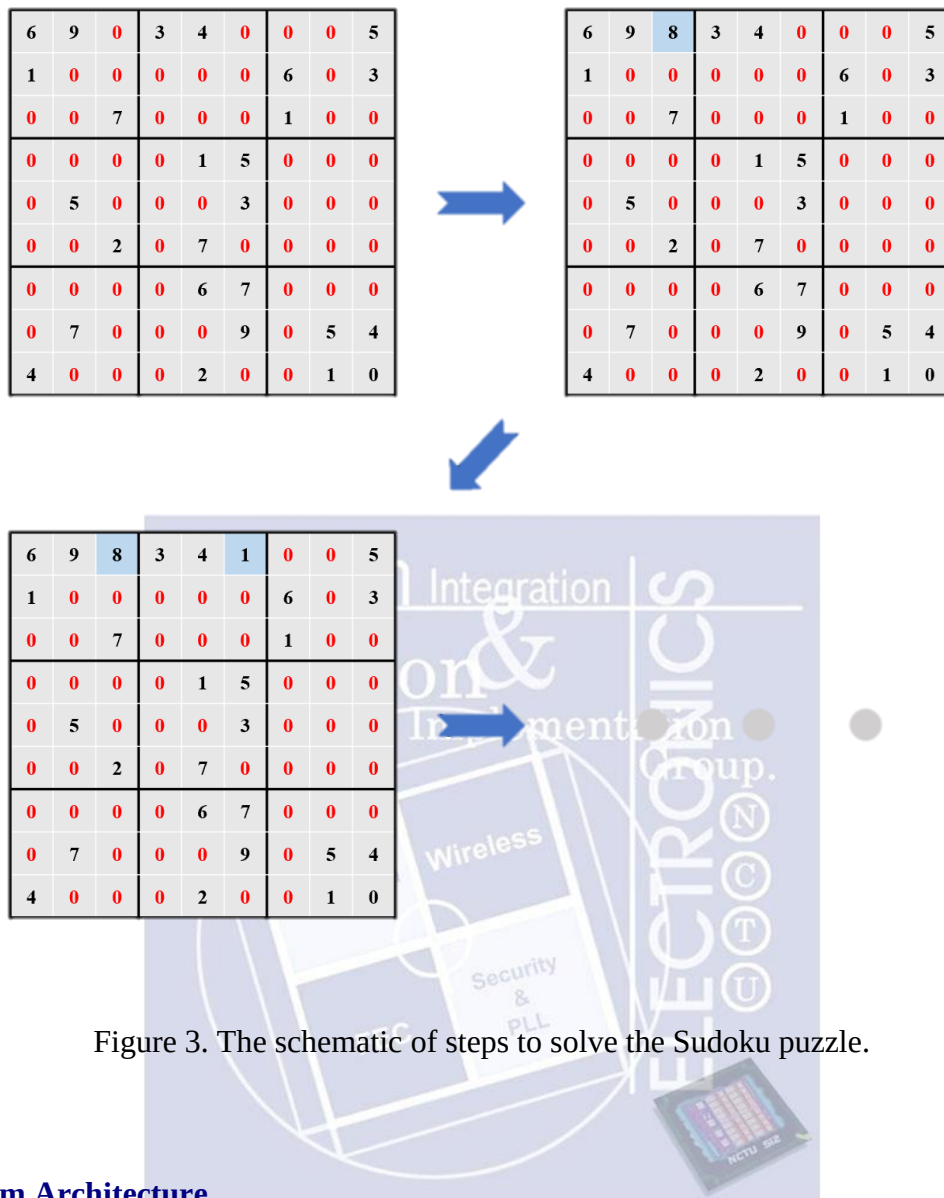
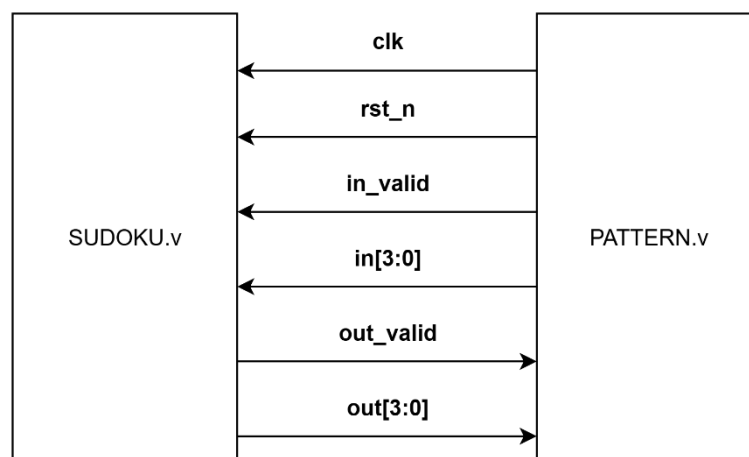


Figure 3. The schematic of steps to solve the Sudoku puzzle.

System Architecture



Inputs

1. When input signal **in_valid** is **high**, the input signal **in** is delivered in **raster scan order** for 81 consecutive cycles.
2. When input signal **in_valid** is **low**, the input signal **in** is tied to unknow (X).
3. All input signals are synchronized at the clock **negative** edge.
4. All operations are **unsigned**.
5. There is **only 1 reset** before the first pattern, thus, your design must be able to reset automatically.
6. The next Sudoku puzzle will come in 3 cycles after **out_valid** is pulled down.

Name	Width	Functional Description
clk	1	Clock for Sudoku
rst_n	1	Asynchronous active-low reset
in_valid	1	When in_valid is high, the input in is valid.
in	4	The digits for Sudoku cell. Range is from 4'd0 to 4'd9. 4'd0: a blank cell 4'd1~4'd9: a pre-filled cell (a given)

Table 1. Input signal information

Outputs

1. When output signal **out_valid** is **high**, the output signal **out** must be delivered in **raster scan order** for 81 consecutive cycles.
2. When output signal **out_valid** is **low**, the output signal **out** is tied to logic-low.
3. All outputs must be synchronized at the clock **positive** edge.
4. The output signal **out_valid** and **out** should be logic-low after initial reset.

Name	Width	Functional Description
out_valid	1	When out_valid is high, the output out is valid.
out	4	The solution to Sudoku puzzle. Range is from 4'd1 to 4'd9.

Table 2. Output signal information

Specifications

1. Top module name: **SUDOKU** (File name: **SUDOKU.v**)
2. It is **asynchronous** reset and **active-low** architecture.
3. The reset signal (**rst_n**) would be given only once at the beginning of simulation.
4. All the data are in **unsigned** format.

5. The output signal **out** should be logic-low when the output signal **out_valid** isn't high.
6. When the output signal **out_valid** is high, the output signal **out** must be correct, and both should be high for 81 consecutive cycles.
7. The total cell area should not larger than **1,200,000 μm^2** .
8. The latency of design in each pattern should not be larger than **1000 cycles**. The latency is the clock cycle between the rising edge of the input signal **in_valid** and the rising edge of the output signal **out_valid**.
9. The maximum clock period is set to **10 ns**, and **you can determine the clock period by yourself**.
10. The input delay and the output delay should be **half** of the clock period, i.e., the input/output delay is set to $0.5 \times (\text{clock period})$.
For example, if clock period is 10 ns, the input/output delay will be 5 ns.
11. The output loading is set to 0.05.
12. **Hardcoding answers** or using a **lookup table** based on the test case index is prohibited; your design must compute outputs correctly based on the actual inputs. You will get 0 point if you use this method, **no second chance**.
13. **Synopsys' IP** and **any comment of // synopsys *** is prohibited in this lab.
14. Only comments in English are allowed.
15. The synthesis result of data type cannot include any **LATCH** (check in syn.log).
16. Don't use System Verilog syntax—if your demo fails due to any of the issues, TAs will not be held responsible.
17. All data flip-flops (D flip-flop/DFF) in your design must be **reset** after the reset signal is asserted.
18. The slack in the end of SUDOKU.timing should be **non-negative** and the result should be **MET**.
19. Note that maximum synthesis time could not be over **10 minutes** (Normally, this design would not over 10 minutes).
20. The gate-level simulation cannot include **any warning of timing violations**.
21. No **ERROR** is allowed in simulation/synthesis log.
22. Don't use any wire / reg / submodule name called "**error**", "**congratulation**", "**latch**" or "**fail**", or you will fail the lab.
23. The performance is determined by **cycle period**, **latency**, and **area**.

Grading Policy

1. 70% for functional validity and 30% for performance

$$\text{Score} = \text{functional validity (70\%)} + \text{performance (30\%)}$$

2. The performance is determined by **cycle period**, **latency**, and **area** of your design.
The smaller the performance index is, the higher score you can get.
$$\text{Perf.} = \text{cycle period} \times \text{latency}^{1.5} \times \text{area}^2$$
3. **Note that cycle period must within 10 ns, or you will fail this lab.**
4. **Note that latency must within 1,000 cycles, or you will fail this lab.**
5. **Note that cell area must within 1,200,000 μm^2 , or you will fail this lab.**
6. Performance points can be obtained if and only if your design passes the TA's patterns.
7. TA will demo your design with the TA's patterns including **the hidden ones**.
8. **The first ten submissions** of designs that pass the TA's pattern will receive **a 5-points bonus**.
9. 2nd demo will only get **50 points**.
10. You will get **0 point** if you are found **plagiarism** at your code; **no second chance**.

Note

1. Please submit your files under 09_SUBMIT before 12:00 at noon:
Due Day: 1st Demo: 9/22 2nd Demo: 9/24
If uploaded files **violate the naming rule**, you will get **5 deduct points**
- In this lab, you can adjust your clock cycle time. Consequently, make sure to key in your clock cycle time after the command like the figure below. It means that the TA will demo your design under this clock cycle time:

```
Exercise/09_SUBMIT]$ ./00_tar 15.0
```

- After that, you should check the following files under **09_SUBMIT/Lab02_iclabXXX/**

RTL design : **SUDOKU_iclab???.v**, (??? is your account number)

Clock period: **clock_period_iclab???.txt (ex: 15.0_iclab999.txt)**

If you miss any files on the list, you will fail this lab.

Then use the command like the figure below to check if the files are uploaded or not.

```
[Exercise/09_SUBMIT]% ./02_check 1st_demo
```

Example:

You will submit **SUDOKU_iclab999.v**, **15.0_iclab999.txt**

If the uploaded files violating the naming rule, you will get **5 deduct points**.

If you omit any file, you will fail demo.

2. Template folders and reference commands:

- 01_RTL/ (for RTL simulation):
./01_run_vcs_rtl
 - 02_SYN/ (for Synthesis):
./01_run_dc_shell
(Check the design which contains **Latch** and **Error** or not in **syn.log**)
(Check the design's timing in /Report/SUDOKU.timing to see if the slack is **MET**)
 - 03_GATE_SIM/ (Gate Level simulation):
./01_run_vcs_gate
 - You can key in ./09_clean_up to clear all log files and dump files in each folder
 - 09_SUBMIT/ (submit your files):
./00_tar your_cycle_period
./01_submit 1st_demo/2nd_demo
./02_check 1st_demo/2nd_demo
3. You should make the clock period values **identical** in 00_TESTBED/PATTERN.v and 02_SYN/syn.tcl
4. You can ignore the warning about TFIPC; however, timing violations is not allowed.

Warning-[TFIPC] Too few instance port connections

```
"/RAID2/COURSE/BackUp/2023_Spring/iclab/iclabta01/umc018/Verilog/umc18_neg.v", 8234: Timing violation in TESTBED.u_SUDOKU.  
$setuphold( posedge CK 666 (flag == 1'b1):819000, posedge D:818882, limits: (175,-77) );
```

A careful person will eventually have good luck.